

Real Estate Agency Portal

CIS344 – Spring 2026

Professor Yanilda Peralta Ramos

Final Project Report

Aysha Akther

1, Project Purpose

Real Estate Agency Portal is an online program created by the use of PHP and MySQL. It is designed to mimic the digital processes of a real estate agency, allowing three kinds of users, namely, agents, buyers, and renters, to interact with property listing and agency service. Buyers and renters can view what is available, save favorites and make enquiries, whereas agents can log in and manage property listing. There is tracking of users, listings, inquiries, favorites, and transactions, offering a realistic end-to-end agency experience.

2. Database Design

This database is referred to as `real_estate_portal_db` and it has five main tables each purposefully created to capture a certain form of entity or relationship within the system.

Users Table

Stores all the registered users of the portal. Every user has a unique user id (`userId`), user name, contact information, user password (hashed, `passwordHash`), and a user type of agent, buyer, or renter. Primary key is the `userId` which auto increases.

Properties Table

Stores real estate advertises. Each property will contain a `propertyId`, title, property type, address, city, price, status (available, sold or rented), and a foreign key `agentId` which will refer to the Users table. This connects all the listing to the agent who made them.

Inquiries Table

Stores the messages relayed by buyers or renters regarding certain properties. Each inquiry has an `inquiryId`, the user id (foreign key to Users), property id (foreign key to Properties), a message and an inquiry date timestamp.

Transactions Table

Documents that were filled contained sales or rental deals. In each transaction, a `transactionId`, `propertyId`, and the user id, the type of the transaction (sale or rental), the date in which the transaction took place, and the amount of the transaction are stored. Both foreign keys are `propertyId` and user id.

Favorites Table

Enables the buyers and rentiers to save the properties that they are interested in. Each record has a `favoriteId`, `userId`, `propertyId` and a `savedDate`. Both, `propertyId` and `userId` refer to their respective tables.

3. PHP Functionality

The application is designed to have a collection of PHP pages and a centralized database class. The fundamental PHP functions used are:

`addProperty()`

Takes in title, `propertytype`, address, city, price, status, and `agentId` as parameters. Adds the new listing on the Properties table. Accessible only to the agents who are logged in.

`addUser()`

Takes: `userName`, `contactInfo`, `passwordHash`, user type. Adds a record to the Users table. Used during registration.

`addInquiry()`

Takes the user id, property id and message. Adds an inquiry to the Inquiries table with the current time as the `inquiryDate`.

`getUserDetails()`

Accepts a user identifier and returns the corresponding user record and associated queries, favorites or transactions. Applied in user dashboard.

Page Structure

The following PHP pages were created to support the platform:

- `index.php` – Homepage with navigation links
- `login.php` – Login page with session creation
- `register.php` – New user registration form
- `properties.php` – Browse all available listings
- `add_property.php` – Agents add new properties
- `property_details.php` – View a single property
- `submit_inquiry.php` – Buyers/renters send inquiries
- `favorites.php` – View saved properties
- `dashboard.php` – User dashboard with role-specific content
- `logout.php` – Destroys session and logs out

4. Login and Session Management.

The system of logging in is incorporated in `login.php`. The system searches the Users table when a user submits his or her credentials with the system searching the table to match a `userName`. The password hash that has been stored is then checked with the built-in `password_verify()` function of the PHP language.

Upon successful login, a PHP session is started and the following session variables are set:

- `$_SESSION['userId']` the ID of the user.
 - `($_SESSION) userName`- the name of the user.
- array: `$_SESSION userType`: the type of user (agent, buyer or renter).

Passwords are encrypted with `password_hash()` with the `PASSWORD_DEFAULT` algorithm (which is currently using `bcrypt`). This would guarantee that the passwords in plain text are never stored in the database.

5. Role-Based Access Control

There is access control in the whole platform depending on the `userType` stored in the session. The actions vary according to position:

Agents

- Has access to `add_property.php` to add new listings.
- Have access to and control their own property listing.
- Is not able to place inquiries or to add favorites.

Buyers and Renters

- Is able to browse properties on properties.php.
- Is able to make inquiries through submit inquiry.php.
- Has the ability to save properties on their favorites list.
- There is no access to property management pages.

Pages with a particular role check the userType of the session at the top of the file. In case the role does not match, the user will be redirected to the home page or an error message of access denial will be displayed.

6. Stored Procedures, View and Trigger.

AddOrUpdateUser (Stored Procedure)

It is a stored procedure that takes as input a userId (optional), userName, contact info, password hash and user type. When a userId is given and already in existence, the record is updated. In case a userId is not given, then a new user is added. This enables the registration and profile update process to be done in a single process.

ProcessTransaction (Stored Procedure)

This is a stored procedure that takes a propertyId, a user id, transaction type, and an amount. It will insert a new record to the Transactions table and then update the corresponding property with its status either sold or rented depending on the type of transaction.

PropertyListingView (View)

This database view is a combination of the Properties and Users tables to provide an easy to use summary of each listing. Columns that are returned include the property title, the property type, city, price, current status and name of the agent. The properties.php uses this view to show listings without having to write a complex JOIN query each time.

AfterTransactionInsert (Trigger)

This is an automatic trigger that is activated once a new row is added to the Transactions table. If the transactionType is 'sale', it sets the property status to 'sold'. When the transactionType is the rent type, then the status becomes the rented status. This means that the property availability is constantly updated without having the application layer to execute another update query.

7. Challenges and Solutions

Challenge: Password Security

The plain-text passwords were being stored in the registration form early in the development process. This was implemented by using password hash () when the form is submitted and password verify () when the user is logging in following the recommended pattern in PHP of dealing with credentials.

Challenge: Page Restrictions (Role-based).

In the absence of a well-defined access control framework, the buyers could initially gain access to agent-only pages by simply going to the URL. This was solved by putting a session check at the top of each restricted page that is reading as \$_SESSION['userType'] and redirecting unauthorized users.

Challenge: Foreign Key Constraints.

The initial failure to insert test data into the Properties and Inquiries tables was because of violation of foreign key constraints. The way to solve this was to always add Users records first,

then Properties, then Inquiries and Transactions, it was necessary to maintain the correct order of insertion.

Challenge: Trigger vs. Stored Procedure Overlap

Status is changed by both ProcessTransaction stored procedure and BeforeTransactionInsert trigger update property. To avoid conflicts, the stored procedure was written to handle only the transaction insertion, leaving the status update entirely to the trigger. This maintains the separation of responsibilities and prevents the occurrence of duplicates updates.

8. Screenshots

Some screenshots of the working platform are provided. These show the home page, the log in page, the property listing view, the add property form, the inquiry submission, the favorites page, and the user dashboard of each role.

9. Conclusion

The Real Estate Agency Portal is a successful implementation of a role-based, database-driven, web-based application with the use of PHP and MySQL. The system addresses all requirements mentioned: users are authenticated prior to granting them access, new users can be registered, agents can manage listings, buyers and renters can browse and interact with properties and all data is tracked through five relational tables. The fact that it uses stored procedures, a view and a trigger shows integration of server-side database logic and the PHP application layer. The secure login system that uses hashed passwords and session management, is a realistic and safe user experience.